

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Bases de datos distribuidas				
NÚMERO DE LA PRÁCTICA:	08	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	09/06/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l8
SOLUCIÓN DE EJERCICIOS PROPUESTOS Ejercicio 1: Caso de estudio FarmaAndes S.A. Enunciado: FarmaAndes S.A. es una cadena farmacéutica con centros de distribución en Arequipa, Lima y Cusco. Cuando una sucursal solicita medicamentos a otra sede, el sistema debe realizar una transacción distribuida para garantizar que el inventario se descuenta del almacén origen, se incrementa en el destino, y que ambas operaciones sean atómicas. Si una operación falla, toda la transacción debe revertirse. Se implementó un sistema de gestión de inventario distribuido para FarmaAndes S.A., coordinando transacciones entre nodos ubicados en Arequipa y Lima mediante el protocolo de confirmación de dos fases (2PC). El diseño se centra en la atomicidad: el descuento en origen y el incremento en destino deben ocurrir ambos o ninguno, evitando estados de inconsistencia donde el stock se pierda o se duplique. Para la persistencia, se utilizaron bases de datos PostgreSQL con restricciones de integridad a nivel de esquema (CHECK stock >= 0). Esto asegura que, incluso si la lógica de aplicación fallara, el motor de base de datos rechazaría transacciones que resulten en stock negativo, disparando un error que el coordinador captura para iniciar un rollback global. <pre>CREATE TABLE inventario (id SERIAL PRIMARY KEY, producto VARCHAR(100) UNIQUE NOT NULL, stock INTEGER NOT NULL CHECK (stock >= 0)); INSERT INTO inventario (producto, stock) VALUES ('Paracetamol', 100);</pre> La lógica de acceso a datos emplea el aislamiento de transacciones mediante bloqueos pesimistas con SELECT FOR UPDATE. Este mecanismo bloquea las filas seleccionadas hasta que la transacción (local) se confirme o aborta, impidiendo que otros procesos modifiquen los mismos registros durante el proceso de preparación del 2PC:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
def lock_and_debit(conn: psycopg.Connection, producto: str, cantidad: int) -> int:
    """
    SELECT FOR UPDATE + UPDATE stock -= cantidad. Retorna stock resultante.
    """
    with conn.cursor() as cur:
        cur.execute(
            "SELECT stock FROM inventario WHERE producto = %s FOR UPDATE", (producto,)
        )
        row = cur.fetchone()
        if row is None:
            raise LookupError(f"Producto '{producto}' no existe en este nodo")
        if row[0] < cantidad:
            raise ValueError(
                f"Stock insuficiente: hay {row[0]}, se requieren {cantidad}"
            )
        cur.execute(
            "UPDATE inventario SET stock = stock - %s WHERE producto = %s",
            (cantidad, producto),
        )
        return row[0] - cantidad
```

El TwoPhaseCommitCoordinator orquesta la transacción en dos etapas críticas. En la Fase 1 (Prepare), el coordinador instruye a cada nodo para que valide la operación y mantenga los cambios en un estado temporal. Si todos los nodos confirman que están listos, el coordinador procede a la Fase 2 (Commit) enviando la orden de confirmación definitiva. Si algún nodo falla en la preparación o pierde conectividad, se ordena un rollback síncrono en todos los participantes:

```
def _phase_one(
    self, txn_id: str, origen: str, destino: str, producto: str, cantidad: int
) -> _PreparedTxn:
    """
    Abre una transacción en cada nodo, aplica los cambios y registra
    la decisión tentativa. Si algo falla, hace rollback en los nodos que
    ya estén abiertos antes de propagar la excepción.
    """
    opened: list[tuple[str, psycopg.Connection]] = []
    self.log.append(txn_id, "VALIDATE", origen, "verificando stock en origen")
    try:
        conn_origen = _open(origen)
    except (pg_errors.Error, psycopg.OperationalError) as e:
        clean_err = db.simplify_db_error(e)
        self.log.append(txn_id, "FAILED", origen, f"error de conexión: {clean_err}")
        raise TransferError(
            f"No se pudo conectar al nodo {origen}: {clean_err}"
        ) from e
    opened.append((origen, conn_origen))
    try:
        stock_origen_pre = db.lock_and_debit(conn_origen, producto, cantidad)
        self.log.append(
            txn_id,
            "PREPARED",
            origen,
            f"debitado {cantidad} (stock pre={stock_origen_pre + cantidad})",
        )
    except (LookupError, ValueError) as e:
        self.log.append(txn_id, "FAILED", origen, f"rechazo en PREPARE: {e}")
        self._rollback_all(opened)
        raise TransferError(str(e)) from e
    # ... (lógica similar para el nodo destino)
```

A continuación se presentan las evidencias de ejecución bajo condiciones normales y de fallo:

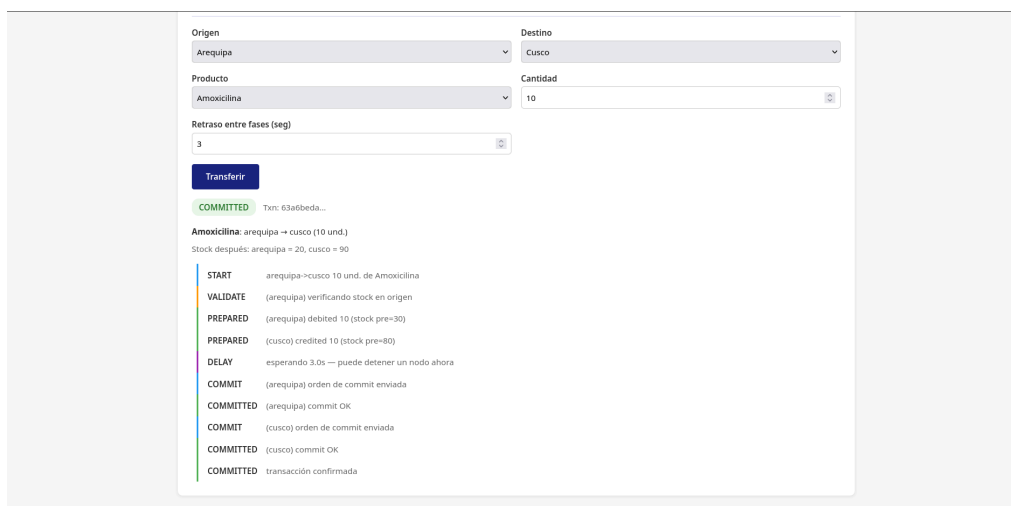


Figura 1: Flujo nominal: transferencia de 20 unidades exitosa entre nodos sincronizados.

La validación de la tolerancia a fallos se realizó deteniendo el servicio de base de datos del nodo destino durante la ventana de tiempo entre fases. El sistema detecta la excepción de red y garantiza que el nodo origen revierta el débito de stock, manteniendo la consistencia global:

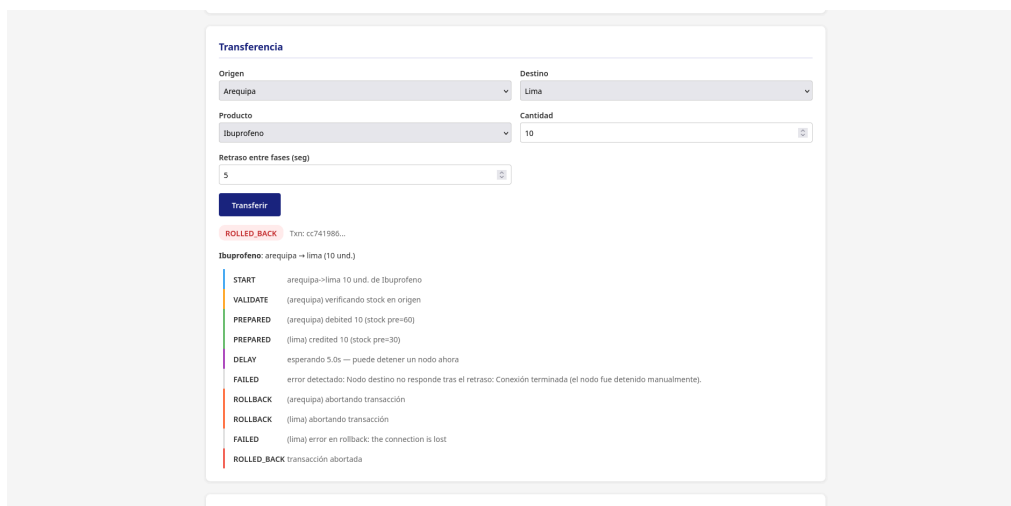


Figura 2: Resiliencia ante fallos: detección de nodo caído y rollback automático en el origen.

Ejercicio 2: Sistema Nacional de Bancos Cooperativos

Enunciado: Una red financiera opera en Arequipa, Cusco y Trujillo. Un cliente solicita transferir S/ 25,000 desde Arequipa hacia Cusco. Se debe diseñar e implementar un modelo distribuido que garantice atomicidad, consistencia y recuperación ante fallos mediante el protocolo Two-Phase Commit (2PC).

El sistema se extendió para manejar transferencias financieras, donde la precisión y la integridad son críticas. Se empleó el tipo de dato DECIMAL (15, 2) de PostgreSQL para evitar errores de redondeo asociados a los tipos de punto flotante. La coordinación ahora gestiona el estado de las transacciones mediante un registro de logs (LogStore) para permitir la trazabilidad de cada fase del protocolo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

La seguridad financiera se garantiza validando los saldos mediante bloqueos de escritura (FOR UPDATE). Esto previene el problema del “doble gasto” al asegurar que ninguna otra transacción pueda leer o modificar el saldo hasta que la transferencia distribuida haya decidido su estado final:

```
def lock_and_debit(conn: psycopg.Connection, numero_cuenta: str, monto: float) -> float:
    """
    SELECT FOR UPDATE + UPDATE saldo -= monto. Retorna saldo resultante.
    """
    monto_decimal = Decimal(str(monto))
    with conn.cursor() as cur:
        cur.execute(
            "SELECT saldo FROM cuentas WHERE numero_cuenta = %s FOR UPDATE",
            (numero_cuenta,),
        )
        row = cur.fetchone()
        if row is None:
            raise LookupError(f"Cuenta '{numero_cuenta}' no existe en esta sucursal")
        if row[0] < monto_decimal:
            raise ValueError(
                f"Saldo insuficiente: hay S/ {float(row[0]):.2f}, se requieren S/ {monto:.2f}"
            )
        cur.execute(
            "UPDATE cuentas SET saldo = saldo - %s WHERE numero_cuenta = %s",
            (monto_decimal, numero_cuenta),
        )
    return float(row[0] - monto_decimal)
```

La implementación del coordinador separa estrictamente la toma de decisiones de la ejecución técnica. En la fase de confirmación, se emplea un plan de ejecución secuencial que verifica la conectividad antes de emitir los comandos de COMMIT finales, minimizando la ventana de vulnerabilidad ante fallos de red de último minuto:

```
def _phase_two_commit(
    self,
    prepared: _PreparedTxn,
    # ... parámetros ...
) -> dict:
    """Ejecuta commit en cada sucursal en orden. Garantiza atomicidad."""
    txn_id = prepared.txn_id
    plan = [
        (ciudad_origen, prepared.conn_origen, "origen"),
        (ciudad_destino, prepared.conn_destino, "destino"),
    ]
    # ... commit secuencial ...
    for ciudad, conn, key in plan:
        self.log.append(txn_id, "COMMIT", ciudad, "orden de commit enviada")
        conn.commit()
        self.log.append(txn_id, "COMMITTED", ciudad, "commit OK")
    # ... procesar respuesta ...
```

Las pruebas demuestran la robustez de la arquitectura en el dominio bancario:

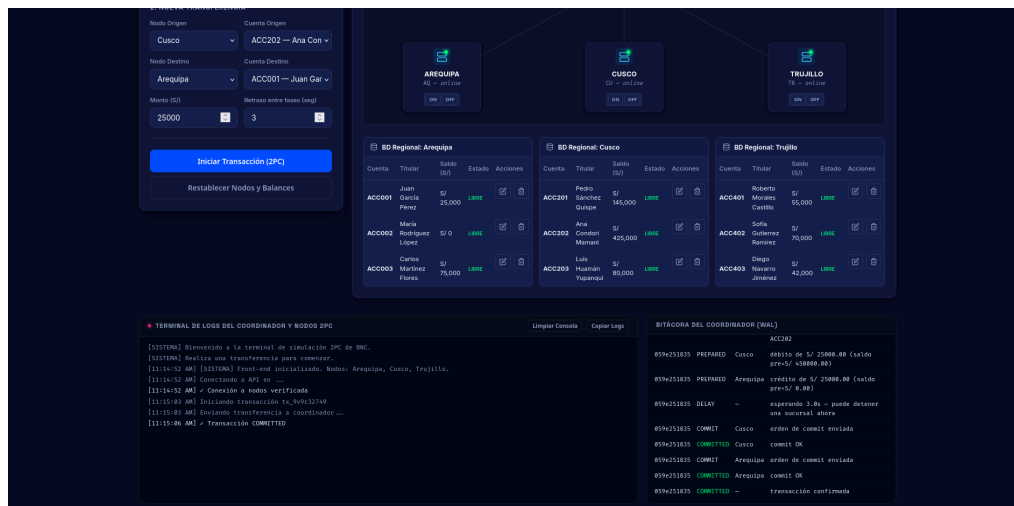


Figura 3: Operación bancaria exitosa de S/ 25,000 con trazabilidad completa de fases.

En la simulación de fallo de red, se observa el comportamiento del coordinador al encontrarse con un nodo inaccesible durante la fase crítica de preparación. Al no recibir la confirmación de "preparado" de Cusco, el sistema aborta la transacción y libera los fondos retenidos en Arequipa, demostrando cumplimiento de las propiedades ACID:

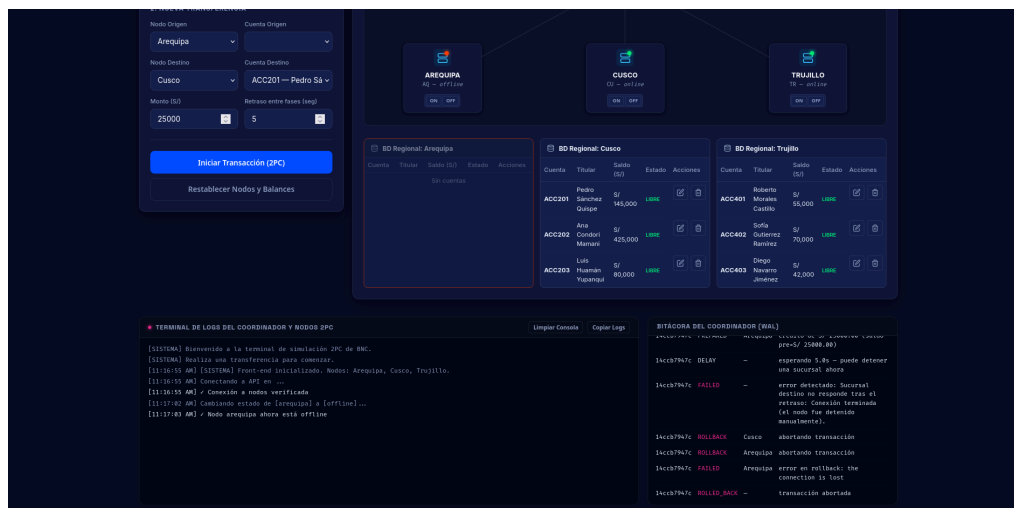


Figura 4: Consistencia financiera: reversión de fondos tras interceptar fallo en nodo remoto.

CUESTIONARIO

1. Una empresa financiera prioriza la disponibilidad del servicio sobre la consistencia de los datos. ¿Qué riesgos podrían surgir y cómo afectarían a los clientes?

Priorizar disponibilidad sobre consistencia (modelo de consistencia eventual) conlleva riesgos significativos como el "doble gasto" o saldos inconsistentes. Un cliente podría retirar dinero simultáneamente desde dos cajeros si los nodos no se han sincronizado, resultando en un saldo negativo no autorizado. Para el cliente, esto genera desconfianza y posibles penalizaciones legales; para el banco, representa pérdidas financieras directas y un caos administrativo en la conciliación de cuentas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

2. El protocolo Two-Phase Commit garantiza consistencia, pero puede reducir la disponibilidad del sistema. ¿Considera que este sacrificio es justificable en todos los contextos empresariales? Fundamente su respuesta.

No es justificable en todos los contextos, pero es imprescindible en el sector financiero y contable. El sacrificio de disponibilidad (bloqueos prolongados si un nodo falla) es el precio a pagar por la integridad absoluta. Sin embargo, en redes sociales o sistemas de inventario de baja criticidad, la consistencia eventual es preferible, ya que un error menor (ej. un "like" que desaparece temporalmente) es menos costoso que tener el sistema caído globalmente por un fallo en un nodo menor.

3. Imagine que una organización global opera cientos de nodos distribuidos. ¿Qué alternativas al protocolo 2PC podrían mejorar el rendimiento sin comprometer significativamente la confiabilidad del sistema?

A escala global (cientos de nodos), 2PC se vuelve ineficiente debido a la latencia acumulada y el riesgo de bloqueos. Alternativas robustas incluyen:

- **Sagas Pattern:** Divide la transacción en pasos compensatorios; si algo falla, se ejecutan transacciones de "anulación".
- **Three-Phase Commit (3PC):** Añade una fase de "pre-commit" para evitar bloqueos indefinidos si el coordinador falla.
- **Protocolos de Consenso (Paxos o Raft):** Utilizados para replicar logs de transacciones de forma tolerante a fallos sin requerir bloqueos estrictos de todos los participantes.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. El protocolo Two-Phase Commit (2PC) es una herramienta fundamental para garantizar las propiedades ACID en sistemas distribuidos, asegurando que los datos permanezcan consistentes incluso ante fallos parciales de red o de nodos.
2. La implementación de bloqueos pesimistas (SELECT FOR UPDATE) a nivel de base de datos es crítica durante la fase de preparación de 2PC para evitar condiciones de carrera que comprometerían la integridad de la transacción global.
3. Existe un compromiso (trade-off) inherente entre consistencia y disponibilidad. Mientras que 2PC garantiza consistencia fuerte, introduce puntos únicos de fallo y latencias que deben ser gestionadas mediante tiempos de espera (timeouts) y logs de recuperación.

RECOMENDACIONES

1. En entornos de alta carga, se recomienda complementar 2PC con mecanismos de monitoreo en tiempo real para detectar nodos "in-doubt" (en duda) y resolver manualmente las transacciones que queden bloqueadas por fallos críticos del coordinador.
2. Es fundamental manejar adecuadamente los tipos de datos (como DECIMAL para dinero) y las restricciones de integridad en el motor de base de datos para que actúen como última línea de defensa ante errores en la lógica de aplicación.
3. Para sistemas con baja tolerancia a la latencia, se sugiere explorar el patrón de Sagas, el cual ofrece una mayor disponibilidad al no requerir bloqueos síncronos sobre múltiples recursos distribuidos.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- [2] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- [3] PostgreSQL Documentation. (2026). Transactions and Concurrency Control. Recuperado de: <https://www.postgresql.org/docs/current/mvcc.html>
- [4] Gray, J., & Reuter, A. (1992). Transaction Processing: Concepts and Techniques. Morgan Kaufmann.